RAJALAKSHMI ENGINEERING COLLEGE

Vignesh V 251001300-2025-IT

V2

Started on	Tuesday, 29 September 2026, 11:54 AM
State	Finished
Completed on	Tuesday, 29 September 2026, 11:55 AM
Time taken	33 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to swap two elements
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function using Lomuto partition scheme
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Choosing the last element as pivot
    int i = (low - 1);    // Index of smaller element

    for (int j = low; j < high; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // pi is partitioning index, arr[pi] is now at right place
        int pi = partition(arr, low, high);

        // Recursively sort elements before partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    // Print the sorted array space-separated
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vignesh V 251001300-2025-IT

V2

Started on	Tuesday, 29 September 2026, 11:53 AM
State	Finished
Completed on	Tuesday, 29 September 2026, 11:54 AM
Time taken	1 min 23 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Divide and Conquer (Binary Search) to find target in range [low, high]
int binarySearch(int arr[], int low, int high, int target) {
    if (low > high) return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target) return mid;
    if (arr[mid] > target) return binarySearch(arr, low, mid - 1, target);
    return binarySearch(arr, mid + 1, high, target);
}

// Divide and Conquer approach to check pair sum in range [low, high]
int findPair(int arr[], int low, int high, int x, int *e1, int *e2) {
    if (low >= high) return 0;

    int mid = low + (high - low) / 2;

    // Case 1: Check if pair exists entirely in the left half
    if (findPair(arr, low, mid, x, e1, e2)) return 1;

    // Case 2: Check if pair exists entirely in the right half
    if (findPair(arr, mid + 1, high, x, e1, e2)) return 1;

    // Case 3: One element in left half [low..mid], check if (x - arr[i]) exists in right half [mid+1..high]
    for (int i = low; i <= mid; i++) {
        int complement = x - arr[i];
        int index = binarySearch(arr, mid + 1, high, complement);
        if (index != -1) {
            *e1 = arr[i];
            *e2 = arr[index];
            return 1;
        }
    }

    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n < 2) {
        printf("No\n");
        return 0;
    }

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vignesh V 251001300-2025-IT V2

Started on	Tuesday, 29 September 2026, 11:52 AM
State	Finished
Completed on	Tuesday, 29 September 2026, 11:53 AM
Time taken	28 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Divide and Conquer approach to find the floor of x
int findFloor(int arr[], int low, int high, int x) {
    int floorIndex = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // If x is present at mid, it is its own floor
        if (arr[mid] == x) {
            return arr[mid];
        }

        // If arr[mid] is smaller than x, it could be a potential floor
        if (arr[mid] < x) {
            floorIndex = mid;      // Store current index as potential floor
            low = mid + 1;         // Search in right half for a larger valid element
        } else {
            high = mid - 1;        // Search in left half
        }
    }

    if (floorIndex != -1) {
        return arr[floorIndex];
    }

    return -1; // Floor does not exist if all elements are > x
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    int floorVal = findFloor(arr, 0, n - 1, x);
    printf("%d\n", floorVal);

    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vignesh V 251001300-2025-IT V2

Started on	Tuesday, 29 September 2026, 11:51 AM
State	Finished
Completed on	Tuesday, 29 September 2026, 11:52 AM
Time taken	1 min 4 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int majorityElement(int* nums, int numsSize) {
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < numsSize; i++) {
        if (count == 0) {
            candidate = nums[i];
        }
        if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    int *nums = (int *)malloc(n * sizeof(int));
    if (nums == NULL) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

```

```
35      printf("%d\n", majorityElement(nums, n));
36
37      free(nums);
38      return 0;
39  }
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vignesh V 251001300-2025-IT

V2

Started on	Tuesday, 29 September 2026, 11:50 AM
State	Finished
Completed on	Tuesday, 29 September 2026, 11:51 AM
Time taken	32 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Function to find the index of the first 0 using Divide and Conquer (Binary Search)
int findFirstZero(int arr[], int low, int high) {
    if (low > high) {
        return -1; // Base case: 0 not found
    }

    int mid = low + (high - low) / 2;

    // Check if the mid element is the first 0
    if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0) {
        return mid;
    }

    // If the element at mid is 1, the first 0 must be in the right half
    if (arr[mid] == 1) {
        return findFirstZero(arr, mid + 1, high);
    }

    // Otherwise, the first 0 must be in the left half
    return findFirstZero(arr, low, mid - 1);
}

// Function to count the number of zeros
int countZeros(int arr[], int n) {
    int firstZeroIndex = findFirstZero(arr, 0, n - 1);

    // If no 0 is present in the array
    if (firstZeroIndex == -1) {
        return 0;
    }

    return n - firstZeroIndex;
}

int main() {
    int m;
    if (scanf("%d", &m) != 1 || m <= 0) return 0;

    int *arr = (int *)malloc(m * sizeof(int));
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    int zerosCount = countZeros(arr, m);
    printf("%d\n", zerosCount);

    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course